

OpenEB 5.2.0 on Windows 11 — build post-mortem

What blocked the build, what fixed it, and how to reproduce the fixes. Environment: Windows 11 26200, VS 2022 17.14 (MSVC 19.44), vcpkg 2024.11.16.

Outcome

Build succeeded. 189 vcpkg packages compiled, 94 libraries produced, OpenEB configured and built (exit code 0). Verified two ways: `metavision_viewer --help` runs, and `import metavision_core` works in the venv.

What actually blocked it

1. CMake 4.4.2 instead of 3.26

The guide specifies CMake 3.26 and notes newer versions may be incompatible. CMake 4.x dropped backward compatibility with old `cmake_minimum_required` declarations that 2024-era vcpkg ports still use. This mismatch was visible early and not acted on. Downgrading to 3.26.6 was correct regardless, though it is unclear whether it was the cause of the Boost failures — vcpkg fetches its own CMake 3.30.1 for port builds, so the system version may not have been in play.

2. Deleting `vcpkg.json` — the largest time sink

The manifest was removed to work around a download error, which switched vcpkg from manifest mode to classic mode and abandoned the pinned baseline. Boost jumped 1.84.0 -> 1.86.0 while its dependencies stayed at 1.84, producing hundreds of constraint-violation warnings and roughly 60 cascading build failures.

Restoring the manifest fixed it. In manifest mode the ports resolve from the pinned registry under `%LOCALAPPDATA%\vcpkg\registries\git-trees`, not from the local ports tree — so the baseline, not the vcpkg checkout, decides package versions.

Lesson: never delete `vcpkg.json` to work around a download failure. The manifest is what makes the build reproducible.

3. Five MSYS2 packages pruned upstream

vcpkg 2024.11.16 hardcodes MSYS2 package URLs with SHA512 hashes in `scripts/cmake/vcpkg_acquire_msys.cmake`. MSYS2 is a rolling release and deletes superseded packages from its mirrors. Twenty-one months after the pin, several of those URLs return 404 on every mirror. This is not machine-specific and is a recurring, documented vcpkg problem.

4. RAV antivirus interference

RAV Endpoint Protection quarantined `perlglob.exe` from Strawberry Perl (a false positive) during the openssl build, and later caused `LNK1168: cannot open h5ls-shared.exe for writing` during hdf5 — the scanner held a lock on a freshly linked binary. Fixed by restoring the file and disabling RAV's real-time protection for the build. Windows Defender had a `C:\vcpkg` exclusion already; RAV's free tier offers no exclusions.

The MSYS2 patch procedure

vcpkg checks `C:\vcpkg\downloads\` before hitting the network. Placing a substitute file there under the expected name, then updating the script's expected hash, makes vcpkg use it. Three parts must line up:

- **Location** — the file goes in the downloads folder vcpkg checks first.
- **Filename** — vcpkg looks up by name, prefixed with the repo: `msys2-<original-pinned-name>`. The name is a label, not a version claim.
- **Hash** — a substitute file has a different SHA512, so the script's expected value is edited to match. This deliberately relaxes an integrity check: the guarantee changes from "exactly the file vcpkg's authors chose" to "the file downloaded from the official MSYS2 repo over HTTPS".

Steps for each package

```
REM 1. find a live version at https://repo.msys2.org/msys/x86_64/  
REM 2. download it, then move + rename to the pinned name
```

```

move "%USERPROFILE%\Downloads\<new-file>" ^
"C:\vcpkg\downloads\msys2-<pinned-name>"

REM 3. compute its SHA512
powershell -c "(Get-FileHash -Algorithm SHA512 ^
'C:\vcpkg\downloads\msys2-<pinned-name>').Hash.ToLower()"

REM 4. find the line to edit
findstr /n /c:"<pinned-name>" ^
C:\vcpkg\scripts\cmake\vcpkg_acquire_msys.cmake

REM 5. edit the SHA512 on the line below the URL, then verify
findstr /c:"<first 8 chars of new hash>" ^
C:\vcpkg\scripts\cmake\vcpkg_acquire_msys.cmake

```

Substitutions made

Pinned (dead)	Substituted	Line
msys2-runtime-3.5.4-2	msys2-runtime-3.5.4-8	458
libtool-2.4.7-4	libtool-2.5.4-1	419
file-5.45-3	file-5.46-1	333
libiconv-1.17-1	libiconv-1.18-1	393
autoconf2.72-2.72-1	autoconf2.72-2.72-3	282

Where possible the nearest available version was chosen — same upstream version with a later build number (runtime, autoconf) carries less risk than a minor-version jump (libtool 2.4.7 -> 2.5.4, libiconv 1.17 -> 1.18). These substitutions are untested by the project and should be the first suspect if an autotools-based port misbehaves.

Note: editing this script changes vcpkg's ABI hash, so every package that touches MSYS2 gets rebuilt on the next run. openssl was recompiled three times as a result (~6-11 min each). Expected, not an error.

Detecting all dead URLs at once

Rather than discovering each 404 by failing a build, this script tests every URL in the acquisition file up front. It found 17 dead of 77 — 5 in the msys set (all needed) and 11 in the mingw64 set (none requested by this dependency tree, so left unpatched).

Save as C:\vcpkg\check-urls.ps1:

```

$script = "C:\vcpkg\scripts\cmake\vcpkg_acquire_msys.cmake"
$lines = Get-Content $script
$dead = @()

for ($i = 0; $i -lt $lines.Count; $i++) {
    if ($lines[$i] -match 'URL\s+(https://mirror\.msys2\.org/[^\s]+)') {
        $url = $matches[1]
        $lineNo = $i + 1
        try {
            $r = Invoke-WebRequest -Uri $url -Method Head -TimeoutSec 20 `
            -UseBasicParsing -ErrorAction Stop
            Write-Host "OK $lineNo $($url.Split('/')[1])"
        } catch {
            Write-Host "DEAD $lineNo $($url.Split('/')[1])" -ForegroundColor Red
            $dead += "$lineNo $url"
        }
    }
}

Write-Host ""
Write-Host "=== DEAD URLS ==="
$dead | ForEach-Object { Write-Host $_ }
Write-Host ""
Write-Host "Total dead: $($dead.Count)"

```

Run with:

```
powershell -ExecutionPolicy Bypass -File C:\vcpkg\check-urls.ps1
```

-UseBasicParsing avoids a per-URL security prompt; -Method Head checks existence without downloading.

Working configuration

- Windows 11 (22H2), NTFS, Win32 long paths enabled
- Git 2.55, CMake **3.26.6**, ffmpeg 9.0.1
- VS 2022 Community 17.14, MSVC 19.44 x64, English language pack, Windows 11 SDK 10.0.22621 (22H2 also installed as a hedge, unused)
- vcpkg tree at tag **2024.11.16**, tool **2024-11-12**, vcpkg.json present
- Python 3.12.10 system; venv at C:\tmp\prophesee\py3venv with requirements_openeb + requirements_pytorch_cpu
- Source at C:\dev\openeb (branch 5.2.0), vcpkg at C:\vcpkg

Build commands

```
cd C:\vcpkg
vcpkg install --triplet x64-windows --x-install-root installed

cd C:\dev\openeb\build
cmake .. -A x64 -DCMAKE_BUILD_TYPE=Release ^
-DCMAKE_TOOLCHAIN_FILE=C:\dev\openeb\cmake\toolchains\vcpkg.cmake ^
-DVCPKG_DIRECTORY=C:\vcpkg -DBUILD_TESTING=OFF
cmake --build . --config Release --parallel 4
```

Running it

The guide's path for the environment script is wrong — utils\scripts\setup_env.bat is a .bat.in template in the source tree. CMake generates the real one into the build folder. Must be run once per Command Prompt:

```
C:\dev\openeb\build\utils\scripts\setup_env.bat
metavision_viewer -i C:\path\to\recording.raw
```

To make it permanent instead: run `cmake --build . --config Release --target install` from an admin prompt, then set PATH, MV_HAL_PLUGIN_PATH and HDF5_PLUGIN_PATH to the C:\Program Files\Prophesee equivalents.

If this recurs on another machine

- Install CMake **3.26.6** specifically, and confirm with `cmake --version` that no newer version shadows it on PATH.
- After cloning vcpkg, always `git checkout 2024.11.16` *before* bootstrap-vcpkg.bat — the bootstrap picks its tool version from the checked-out tree.
- Copy vcpkg-openeb.json to vcpkg.json and never remove it. Skip vcpkg update; it does nothing in manifest mode and preceded the baseline drift here.
- Run the URL checker before the first long build. Expect the dead set to have grown — MSYS2 keeps pruning.
- Disable real-time antivirus scanning (all products, not just Defender) for the duration of the build.
- Note the substitutions above are not upstream fixes. A fresh vcpkg checkout resets vcpkg_acquire_msys.cmake and they must be redone. A .bak of the patched file is worth keeping.